Projektarbeit: E-Commerce REST-API (Golang)

Dieses Dokument beschreibt die Architektur, die Geschäftslogik und die Datenstrukturen einer E-Commerce-Plattform.

1. Architektur-Prinzipien

- **Sprache:** Go (Golang)
- **Entwurfsmuster:** Schichtenarchitektur, 1 Modul, 1 Package per Use Case
- **Datenbank:** PostgreSQL (relationale Speicherung) mit GORM https://gorm.io/docs/index.html
- **Authentifizierung:** Stateful Sessions (tokenbasiert mit Datenbank-Validierung).
- **Header-Check:** Alle Anfragen an Endpunkte mit einer anderen Rolle als Public **müssen** den Session-Token im HTTP-Header ***X-Session-Token*** mitsenden. Der Server validiert diesen Token bei jedem Request gegen die sessions–Tabelle
- **Passwort-Sicherheit:** Obligatorische Verwendung von bcrypt für das Hashing vor dem Speichern.
- **Identität:** UUIDs für alle Primärschlüssel. https://github.com/lithammer/shortuuid
- **Datenstrukturen:** Keine Verwendung von JSONB. Alle Daten werden in flachen, expliziten Tabellenspalten gespeichert.
- Testfall für jeden Endpoint erstellen
- Code-Coverage soll mindestens 90 % sein
- Dokumentation mit Swagger UI (swaggo).

2. Bounded Contexts

A. Identity & Access Management (IAM)

Logik: Passwörter werden mit bcrypt gehasht. Ein erfolgreicher Login erzeugt einen Eintrag in der sessions-Tabelle.

POST /auth/register

- **Rolle:** –
- **Request Body:**
  ```
  {
    "username": "max_mustermann",
    "email": "max@example.com",
    "password": "sicheres_passwort_123"
  }
  ```
- **Response (201 Created):** {"id": "uuid", "username": "string", "email": "string"}
- **Involvierte Datenbank-Tabellen:** * users (INSERT): Erstellung des neuen Benutzerdatensatzes mit password_hash.
- Hint: Nutzen Sie bitte bcrypt dafür https://pkg.go.dev/github.com/absagar/go-bcrypt#section-readme

POST /auth/login

- **Rolle:** –
- **Request Body:** {"email": "string", "password": "string"}
- **Response (200 OK):** {"session_token": "string", "expires_at": "timestamp", "user": {"username": "..", role: "Customer"}}
- **Involvierte Datenbank-Tabellen:** * users (SELECT): Abruf des Hash-Wertes zum Vergleich via bcrypt.

- sessions (INSERT): Erstellung eines neuen Session-Eintrags bei erfolgreichem Login.

POST /auth/logout

- **Rolle:** Customer / Admin (**Header erforderlich**)
- **Response (204 No Content)**
- **Involvierte Datenbank-Tabellen:** sessions (DELETE): Entfernen des Tokens aus der Datenbank.

B. Product Catalog

GET /products

- **Rolle:** -
- **Query-Parameter:** category_id, page(1 page = 10 Elemente)
- **Response (200 OK):** {"items": [{"name": "string", "price": 89.90, category_id:".."}, ...], "total_count": 120}
- **Involvierte Datenbank-Tabellen:** products (SELECT): Abruf der Produktdaten (ggf. gefiltert nach is_active = true).
  - categories (SELECT): Optionaler Join oder Prüfung der Kategorie-Existenz.

POST /products

- **Rolle:** Admin (**Header erforderlich**)
- **Request Body:** {"name": "string", "price": 89.90, description:"..", category_id: "..."}
- **Response (201 Created)**
- **Involvierte Datenbank-Tabellen:** products (INSERT): Anlage des neuen Stammdatensatzes.
  - sessions (SELECT): Validierung des Admin-Status via Middleware.

C. Inventory (Lagerbestand)

GET /inventory/{product_id}

- **Rolle:** Customer / Admin (**Header erforderlich**)
- **Involvierte Datenbank-Tabellen:** stocks (SELECT): Abruf von Menge, reserviertem Bestand und Lagerort.

PATCH /inventory/{product_id}

- **Rolle:** Admin (**Header erforderlich**)
- **Request Body:** {"increment_by": 20, "reason": "string"}
- **Involvierte Datenbank-Tabellen:** stocks (UPDATE): Erhöhung der Spalte quantity.

D. Shopping Cart (Warenkorb)

GET /cart

- **Rolle:** Customer (**Header erforderlich**)
- **Involvierte Datenbank-Tabellen:** carts (SELECT): Abruf des aktiven Warenkorbs des Users.
  - cart_items (SELECT/JOIN): Abruf der Produkte im Warenkorb inklusive Preisen aus der products Tabelle.

POST /cart/items

- **Rolle:** Customer (**Header erforderlich**)
- **Request Body:** {"product_id": "uuid", "quantity": 1}
- **Involvierte Datenbank-Tabellen:** carts (SELECT/INSERT): Prüfung, ob bereits ein Warenkorb existiert, andernfalls Erstellung.

- cart_items (INSERT/UPDATE): Hinzufügen des Produkts oder Erhöhen der Menge bei existierendem.

E. Ordering (Bestellwesen)

POST /orders

- **Rolle:** Customer (**Header erforderlich**)
- **Request Body:**
  ```
  {
    "items": [{ "product_id": "uuid", "quantity": 2 }],
    "shipping_street": "string",
    "shipping_city": "string",
    "shipping_zip": "string",
    "shipping_country": "string"
  }
  ```
- **Involvierte Datenbank-Tabellen:**
  - orders (INSERT): Erstellung des Kopfdatensatzes mit flachen Adressfeldern.
  - order_items (INSERT): Speichern der Positionen mit aktuellem unit_price.
  - stocks (UPDATE): Erhöhung der Spalte reserved oder Verringerung von quantity.
  - cart_items (DELETE): Leeren des Warenkorbs nach erfolgreicher Bestellung.

F. Notifications

GET /notifications

- **Rolle:** Customer (**Header erforderlich**)
- **Involvierte Datenbank-Tabellen:** * notifications (SELECT): Abruf der Nachrichten für die eigene user_id.

PATCH /notifications/{id}/read

- **Rolle:** Customer (**Header erforderlich**)
- **Involvierte Datenbank-Tabellen:** * notifications (UPDATE): Setzen des Flags is_read = true für die spezifische ID.

Ein Notification-Eintrag soll eingelegt werden, wenn folgende Aktionen ausgeführt werden: Register, Login, Logout, Produkt anlegen, Lagerbestand und Warenkorb aktualisieren.

3. Datenbank-Detaillierung (Schema)

Tabelle: users

Feld	Typ	Beschreibung
id	UUID (PK)	Eindeutige ID.
username	VARCHAR	Anzeigename.
email	VARCHAR	Unique Email für Login.
password_hash	TEXT	Bcrypt Hash.
role	VARCHAR	'admin' oder 'customer'.

Tabelle: sessions

Feld	Typ	Beschreibung
id	UUID (PK)	Eindeutige ID.
user_id	UUID (FK)	Referenz auf users.
token	VARCHAR	Einzigartiger Session-String.
expires_at	TIMESTAMP	Ablaufdatum der Session.

Tabelle: orders

Feld	Typ	Beschreibung
id	UUID (PK)	Bestellnummer.
user_id	UUID (FK)	Käufer.
status	VARCHAR	z.B. 'placed', 'shipped'.
total_amount	DECIMAL	Gesamtsumme.
shipping_street	VARCHAR	Adresse: Straße.
shipping_city	VARCHAR	Adresse: Stadt.
shipping_zip	VARCHAR	Adresse: PLZ.
shipping_country	VARCHAR	Adresse: Land.

Weitere Tabellen (Zusammenfassung)

- **products**: id, name, description, price, category_id, is_active.
- **stocks**: product_id (FK), quantity, reserved, location.
- **order_items**: order_id (FK), product_id (FK), quantity, unit_price.
- **cart_items**: cart_id (FK), product_id (FK), quantity.
- **notifications**: id, user_id (FK), title, message, is_read.

4. Standard HTTP Status Codes

- **200 OK:** Erfolgreiche Abfrage.
- **201 Created:** Ressource erfolgreich angelegt.
- **400 Bad Request:** Validierungsfehler (z.B. ungültiges JSON).
- **401 Unauthorized:** Token fehlt oder ist ungültig.
- **403 Forbidden:** Fehlende Rechte (z.B. User möchte Admin-Aktion ausführen).
- **404 Not Found:** Ressource existiert nicht.
- **500 Internal Server Error:** Unerwarteter Serverfehler.

Ihre Abgabe soll im ZIP-Format am Donnerstag, **15. 01. 2026 um 15:35 Uhr** hochgeladen werden